MODULE 18 · DOMAIN 5 - MODEL SELECTION AND OPTIMISATION

Choosing the Model

Fable, Opus, Sonnet and Haiku as tiers; the quality, latency and cost triangle; a selection sequence built on your own evals; routing between models; and surviving new releases.

DOMAIN 5 of 8 EXAM WEIGHT 16.8% MODULE 18 of 36

admin@sthithapragna.com

You staff a job, not a CV. Pick the least expensive model that passes your evals, and prove it with your own data.

The current tiers

MODEL	USD PER MTOK IN / OUT	WHEN YOU NEED
Fable 5.1	10 / 50	The highest capability: hours-long agents, deep research
Opus 5.5	4 / 20	Complex agentic coding and enterprise work
Sonnet 5.5	2 / 10	Everyday coding, agents and enterprise workloads
Haiku 5.5	0.10 / 0.50	Lowest latency and price, high volume, subagents

ON THE EXAM

All four: 1M-token context, 128K output, output priced at 5x input. Haiku 5.5 costs more above 100K-token prompts.

admin@sthithapragna.com

The triangle: quality, latency, cost

- **Quality:** does it pass your success criteria on representative inputs?
- **Latency:** time to first token and total time, against the requirement.
- **Cost:** per unit of work, at your real volume, including thinking and output.
- **Two dials, not one:** the model sets the ceiling; effort trades depth for speed and tokens within it.
- **Fast mode** buys output speed on Opus 5.5, at a premium.

> THE TIEBREAKER
>
> Name the corner the requirement fixes first. The other two are then traded against it.

The selection sequence

01
Requirements
Criteria for quality, latency, cost, data. Module 9.

02
Candidates
Two or three models, at two or three effort levels.

03
Test
Your eval set, on your real prompts and data.

04
Decide
Least expensive option that passes every criterion.

05
Monitor
Quality and cost in production; revisit on new releases.

ON THE EXAM

Anthropic: "Having a good evaluation set is the most important step in the process."

Two ways to start

EFFICIENCY-FIRST

Start with Haiku 5.5

- Implement and test thoroughly
- Check it meets the requirements
- Upgrade only for specific capability gaps
- Suits high-volume, latency-sensitive work

CAPABILITY-FIRST

Start with Opus 5.5

- Optimise prompts on the strong model
- Check it meets the requirements
- Then lower effort or downgrade
- Fable 5.1 if xhigh or max still fall short

THE TAKEAWAY

Both routes end in the same place: the evals decide, not the starting point.

Routing: more than one model

- **Route by task:** code or a small classifier sends easy work to a cheaper model, hard work up.
- **Executor and advisor:** a lower-cost model works, and escalates hard decisions to a frontier model.
- **Orchestrator and workers:** a strong model plans and delegates bulk work to cheaper workers.
- **Same evals per route:** each path must pass the criteria for the work it receives.
- **Routing adds complexity.** Use it when volume and the mix of difficulty justify it.

> CURRENCY CHECK
>
> Anthropic: multi-model strategies bill "most tokens … at the lower rate".

Copperlet's model map

FEATURE	MODEL AND EFFORT	WHY
Tenant help assistant	Sonnet 5.5, medium	Chat: quality with low latency
Repair triage	Sonnet 5.5, medium	Passed R1 and R2; Haiku missed emergencies
Overnight sweep notes	Haiku 5.5, in a batch	Simple, high volume, nobody waiting
Agency-ops agent	Opus 5.5, high	Multi-step investigation across systems
Document reader	Sonnet 5.5, high	Long documents, citations, accuracy

THE TAKEAWAY

Every row was chosen by evals against that feature's requirements, and is revisited on new releases.

What changes between releases

MOVING TO SONNET 5.5 FROM 4.X	WHAT IT MEANS FOR YOU
Thinking on by default; text omitted	Revisit max_tokens and cost; set display to show it
About 30% more tokens for the same text	Recount tokens; re-baseline cost
Effort levels recalibrated; default high	Re-run an effort sweep at two or three levels
Sampling, prefill, forced tool choice: 400	Remove or replace them before switching
Declines in more refusal categories	Handle refusals and configure a fallback

ON THE EXAM

Same prices per token can still mean a higher bill. Measure, do not assume.

Surviving a new release

01
Read
The model's migration guide and checklist.

02
Fix
Breaking changes in code and requests.

03
Evaluate
Regression set and effort sweep; re-baseline cost.

04
Adapt
Prompts, against the new model's prompting guide.

05
Canary
A flag, a small share of traffic, then expand.

THE TIEBREAKER

Read, fix, evaluate, adapt, then upgrade. Changing the model ID is the last step, not the first.

Copperlet: the Friday upgrade

BEFORE

Swap and hope

- Triage model ID changed on a Friday afternoon
- No eval run; no migration guide read
- Monday: labels in a new format
- Agents re-triaged tickets by hand

AFTER

A model change is a release

- Migration checklist in the pull request
- 200-ticket set and effort sweep must pass
- Cost re-baselined on recounted tokens
- Canary by flag, then expand over days

SETTLED

Quirk 13 settled. The move to Sonnet 5.5 was needed anyway: Sonnet 4.5 retires on 30 November 2026.

> An online retailer's order-status chatbot answers short, simple questions at very high volume. Responses must start quickly. The prototype uses Fable 5.1 and meets every quality criterion, but the projected bill is far over budget.
>
> No one has tested any other model.

What should the team do?

A Keep Fable, since it already passes every quality criterion

B Test smaller, faster models on the eval set, and choose the least expensive one that passes

C Reduce max_tokens on Fable until the projected cost fits

D Request a discount, and keep the current design

◆ PAUSE AND DECIDE

What should the team do?

A Keep Fable, since it already passes every quality criterion

Quality is one corner; cost fails.

B Test smaller, faster models on the eval set, and choose the least expensive one that passes

Evidence picks the right tier.

C Reduce max_tokens on Fable until the projected cost fits

Truncates answers; the rate stays high.

D Request a discount, and keep the current design

Ignores the mismatch of tier and task.

THE SIGNAL

"Simple, high volume, must start quickly" and "no one has tested any other model": test down.

> An engineering team uses Sonnet 5.5 at medium effort for an autonomous agent that refactors large codebases over several hours. On their evals it often loses track of the plan and leaves the refactor unfinished.
>
> Quality matters far more than cost for this work.

What should the team try first?

A Lower the effort to low, so that the agent finishes faster

B Move the agent to Haiku, which handles subagent tasks

C Add more few-shot examples of finished refactors to the prompt

D Evaluate Opus and higher effort levels on the same refactoring evals

◆ PAUSE AND DECIDE

What should the team try first?

A Lower the effort to low, so that the agent finishes faster

Less depth, on a task short of depth.

B Move the agent to Haiku, which handles subagent tasks

Down a tier, the wrong direction.

C Add more few-shot examples of finished refactors to the prompt

Long examples, little help with long-horizon work.

D Evaluate Opus and higher effort levels on the same refactoring evals

The tier built for this work, tested.

THE SIGNAL

"Hours-long agentic coding" and "quality over cost": test up a tier and up the effort dial.

> A team chose a model because it topped a public benchmark leaderboard. On their own insurance documents, extraction accuracy is disappointing.
>
> They are now considering another leaderboard.

What should they do?

A. Build an eval set from their own documents and compare candidates on it

B. Pick the model ranked first on a different leaderboard

C. Keep the current model, since benchmarks show it is the strongest

D. Switch to whichever model is newest

◆ PAUSE AND DECIDE

What should they do?

A Build an eval set from their own documents and compare candidates on it

Their task, their data, their criteria.

B Pick the model ranked first on a different leaderboard

The same mistake, again.

C Keep the current model, since benchmarks show it is the strongest

Their own results say otherwise.

D Switch to whichever model is newest

Newest is not tested.

THE SIGNAL

Public benchmarks measure someone else's task. Your eval set measures yours.

> A support platform handles 50,000 requests a day. About 85% are simple account questions; about 15% need multi-step reasoning across policies. Everything currently runs on Opus 5.5, and costs are high.
>
> Quality on the hard requests must not drop.

What is the best design?

A Move everything to Haiku, and accept lower quality on hard requests

B Keep everything on Opus, since some requests need it

C Route simple requests to a cheaper model and hard ones to Opus

D Let each customer choose which model answers their question

◆ PAUSE AND DECIDE

What is the best design?

A Move everything to Haiku, and accept lower quality on hard requests

Breaks the stated quality requirement.

B Keep everything on Opus, since some requests need it

Pays frontier rates for simple work.

C Route simple requests to a cheaper model and hard ones to Opus

Most tokens at the lower rate; quality kept.

D Let each customer choose which model answers their question

Customers cannot judge difficulty.

THE SIGNAL

A mix of easy and hard work at volume, with a quality floor on the hard part: route.

> A team will move a production feature from Sonnet 4.5 to Sonnet 5.5. Its requests use an assistant prefill and set temperature, and its cost budget is tight.
>
> The team wants the switch to go smoothly.

Which two steps should come before switching traffic? Choose two.

A. Change the model ID on Friday, and watch for complaints over the weekend

B. Keep temperature in the requests, to hold outputs consistent

C. Follow the migration guide, replacing the prefill and removing sampling parameters

D. Assume costs are unchanged, since the price per token is similar

E. Re-run the evals at two or three effort levels, and re-baseline cost on recounted tokens

◆ PAUSE AND DECIDE

Which two steps should come before switching traffic? Choose two.

A Change the model ID on Friday, and watch for complaints over the weekend

Copperlet's quirk 13.

B Keep temperature in the requests, to hold outputs consistent

Returns an error on Sonnet 5.5.

C Follow the migration guide, replacing the prefill and removing sampling parameters

Fixes the breaking changes first.

D Assume costs are unchanged, since the price per token is similar

More tokens, thinking on by default.

E Re-run the evals at two or three effort levels, and re-baseline cost on recounted tokens

Evidence on quality and cost.

CURRENCY CHECK

Prefill and temperature are breaking changes; a tight budget needs recounted tokens.

> After moving a summarisation service from an older Sonnet to Sonnet 5.5 at similar per-token prices, the monthly bill rose by roughly a third. Request volume and documents did not change.
>
> Finance asks why.

What is the most likely explanation?

A. A billing error, to be raised with support

B. The same text now produces more tokens, and thinking runs by default

C. Prompt caching stopped working after the switch

D. Rate limits forced extra retries on every request

◆ PAUSE AND DECIDE

What is the most likely explanation?

A. A billing error, to be raised with support
The cause is documented, not an error.

B. The same text now produces more tokens, and thinking runs by default
About 30% more tokens, plus thinking.

C. Prompt caching stopped working after the switch
Nothing points to it; the minimum fell.

D. Rate limits forced extra retries on every request
Failed retries are not billed as output.

THE SIGNAL

Same price, same volume, bigger bill: count the tokens, not the price list.

> A chat assistant moved to a newer model at its default effort of high. Answers are good, but users now wait noticeably longer for each reply, and the latency requirement is being missed.
>
> Responses are already streamed.

What should the team try?

A Move to Fable, the most capable model

B Send the chat requests through the Message Batches API

C Turn off streaming, so replies arrive in one piece

D Evaluate medium or low effort for this chat workload, checking quality still passes

◆ PAUSE AND DECIDE

What should the team try?

A Move to Fable, the most capable model

More capability, not less latency.

B Send the chat requests through the Message Batches API

A person is waiting.

C Turn off streaming, so replies arrive in one piece

Makes the wait feel longer.

D Evaluate medium or low effort for this chat workload, checking quality still passes

Anthropic's advice for chat latency.

CURRENCY CHECK

Default effort is a starting point. Latency-sensitive chat usually starts at medium or low.

> A team's feature runs on claude-sonnet-4-5-20250929, which is deprecated and retires on 30 November 2026. The team wants to stay on it because its prompts are finely tuned.
>
> Sonnet 5.5 is the named replacement.

What should the team do?

A Migrate to the replacement before the date, with the guide and evals

B Keep the current ID, since requests to retired models are redirected automatically

C Pin the dated ID more firmly in configuration, so it cannot change

D Wait for the retirement date, then decide

◆ PAUSE AND DECIDE

What should the team do?

A Migrate to the replacement before the date, with the guide and evals

Planned, tested, on time.

B Keep the current ID, since requests to retired models are redirected automatically

They fail, not redirect.

C Pin the dated ID more firmly in configuration, so it cannot change

Pinning does not stop retirement.

D Wait for the retirement date, then decide

Requests fail that day.

THE SIGNAL

Pinning protects you from surprise changes. Only migration protects you from retirement.

Module 18 in five lines

- **Tiers:** Fable for the hardest work, Opus for complex agents, Sonnet everyday, Haiku for speed and volume.
- **Trade quality, latency and cost;** effort is a second dial inside each model.
- **Your eval set decides:** least expensive option that passes every criterion.
- **Route mixed traffic** when volume justifies it, with evals per route.
- **New release:** read, fix, evaluate, adapt, canary. The ID changes last.

COMING UP

Next: cost and token management - budgets, tracking and caching.